

Engineering Efficient Self-Play Chess: Search, Replay, and Throughput Under Limited Compute

Bertil Braun

contact@bertil-braun.de

Abstract

How strong can an AlphaZero-style chess system become under limited training compute when its entire learning loop is engineered for efficiency? We train from random initialization through searched self-play on a single eight-GPU node for 2.5 days. The resulting 6.32-million-parameter model reaches 3,251 benchmark Elo [3,206, 3,297] at 100,000 searches per move (roughly five seconds of thinking time) against a fixed-node Stockfish 13 ladder. The run ingests 3.25 million completed games, involves roughly 195 billion search simulations, most requiring neural-network evaluation, and makes 836.6 million training presentations. We investigate search allocation, replay and restart-state selection, policy representation, progressive model sizing, quantized inference, and throughput engineering. Alongside the retained design, we document plausible alternatives that failed to improve the complete learning loop or did not justify their cost. The reported strength is a result of the integrated system, not an isolated Elo gain attributable to any single choice.

Keywords: AlphaZero, chess, self-play, Monte Carlo tree search, replay, GPU inference, quantization, model growth, fixed-node evaluation

1 How far can efficient self-play go?

An AlphaZero-style chess player [1] improves by searching its own games, learning from the resulting positions, and repeating that cycle with a stronger network. Search creates targets; replay decides which targets persist; training absorbs them; and evaluation must distinguish progress from noise. Under a limited budget, improving any one step matters only if the complete loop produces stronger play within the available time.

This study asks how strong that loop can become on a single eight-GPU node when the system is engineered for efficiency. Over **2.5 days** of training from random weights on searched self-play games, the run produced a 6.32-million-parameter model. It reached **1,658 benchmark Elo without search** and **3,251 benchmark Elo at 100,000 searches per move**—roughly five seconds of thinking time—against the project’s fixed-node Stockfish 13 ladder. Table 1 gives the match results; Chapter 8 follows the model’s improvement across search budgets.

The result depends on the whole learning loop. The network’s representation affects both what it can learn and how quickly it can supply search. Native search, batched inference, and TensorRT make enough searched games available for training; replay and training must then turn those games into stronger play. Several plausible optimizations improved a local metric without improving that complete loop.

1.1 Scope and contributions

Chess is the subject of the study. The runtime also supports Go, and KataGo’s fast and full searches inspired one of the approaches tested here [7]. We briefly considered 7×7 and 9×9 Go as cheaper settings for tuning chess hyperparameters. In those exploratory games, we observed a strong first-player advantage, short trajectories, and a value target that learned quickly. Useful Go tuning appeared unlikely to transfer directly to chess, so we kept chess as the focus.

We examine the choices that shaped the final recipe: search

allocation, graph search and caching, policy representation, model sizing, replay and restart states, resignation, auxiliary targets, and quantized inference. The report also traces the throughput needed to supply searched games and studies three failures with transferable lessons about self-play targets, deployment fidelity, and model promotion.

A live chess demonstration is also available [12].

1.2 Roadmap

Chapter 2 presents the chess evaluation, and Chapter 3 introduces the system. The following chapters examine search, replay, network design, and the throughput needed to train them. Three failure studies lead into the final recipe and evaluation.

2 Evaluating the chess system

The most direct test of the system is whether its final model wins games. We played it against Stockfish 13 at fixed search limits, starting from 50 openings and playing each once with each colour. The model played without search and at four progressively larger search budgets. For each budget, we tested two Stockfish limits rather than trusting a single opponent.

2.1 Final playing strength

Table 1 gives all ten matches. Each row contains 100 games; wins, draws, and losses are from our model’s perspective. The bold rating for each model budget comes from the opponent against which it scored closest to 50%, where the rating estimate needs the least extrapolation.

Table 1: Final checkpoint against fixed-node Stockfish 13

Model searches	Parallel	Stockfish nodes	W/D/L	Score	Benchmark Elo (95% CI)
Policy only	–	1,000	32/24/44	0.440	1,658 [1,608, 1,710]
Policy only	–	2,000	16/22/62	0.270	1,717 [1,638, 1,790]
100	1	5,000	51/28/21	0.650	2,328 [2,276, 2,384]
100	1	10,000	39/18/43	0.480	2,456 [2,400, 2,512]
1,000	1	20,000	47/40/13	0.670	2,823 [2,774, 2,873]
1,000	1	50,000	21/48/31	0.450	2,925 [2,875, 2,977]
10,000	4	50,000	45/40/15	0.650	3,068 [3,023, 3,120]
10,000	4	100,000	30/44/26	0.520	3,114 [3,065, 3,163]
100,000	16	100,000	51/38/11	0.700	3,247 [3,192, 3,305]
100,000	16	200,000	25/56/19	0.530	3,251 [3,206, 3,297]

At the deepest budget, the model scored 3,251 benchmark Elo against the harder opponent and 3,247 against the other: the two measurements agree closely. The ratings use a published calibration of Stockfish’s fixed node limits [9]. Chapter 8 examines how strength changes with search; Appendix B gives the rating calculation and interval method.

2.2 Reading the component experiments

The rest of the paper asks why the system reached that result. A faster inference engine can supply more search, but that only helps training if the system finishes more games and admits useful positions to replay. A lower loss on stored positions can indicate a better fit, but the real test is whether the next model plays better. We follow each proposed improvement as far through this chain as the experiment measured.

For online comparisons, we matched starting weights, replay, evaluation, and hardware where possible. Smaller frozen-data tests helped screen ideas before expensive self-play runs. Because the final recipe combines many changes, we attribute a separate strength gain to a component only when a comparison actually measured one.

3 System and training method

3.1 One learning cycle

The system improves by repeatedly turning played games into a stronger player. A published network guides search in many self-play games; the resulting positions and search policies enter replay; the trainer learns from that replay and publishes the next network. Evaluation checks whether the new player is actually stronger. Figure 1 shows this cycle and the components that carry it. Chapter 5 asks which parts of the cycle limit how much learning can happen in a fixed time.

Native C++ plays the games and runs search so Python does not handle every position or tree operation. Python coordinates workers, replay, training, publication, and evaluation. Both sides must interpret chess positions and network outputs identically; tests check their action mapping, features, symmetries, tensor shapes, and output order.

3.2 Chess representation and outputs

The network sees the board, recent history, and rule-relevant state. It predicts a move policy and the probabilities of winning, drawing, or losing. Search uses those two outputs to choose moves and to create training targets. The retained model shares a convolutional representation between its outputs and scores moves by origin and destination; training-only

auxiliary heads are removed from the serving artifact. Section 4.3 explains the representation choices, including the 1,880-action interface; Chapter 7 gives the selected model shape.

3.3 Search and self-play

Self-play searches each played position with neural policy priors and value estimates. The visit budget grows in stages as training progresses, but is fixed across positions within a stage. Each search produces both a move and a policy target for learning. Section 4.1 explains the search rules and why the tested ways of varying the budget within a stage were not retained.

Many games run concurrently so their neural evaluations can share GPU batches. Native workers retain search trees between moves, and TensorRT serves the batches. Searching several leaves of one tree at once can fill a batch when fewer independent games are ready, but the leaves then see partly stale search information. Sections 4.1 and 5 measure that quality-throughput tradeoff and the worker topology.

Games start either from shallow random openings or from archived self-play positions worth revisiting. This keeps ordinary opening-to-endgame games in the stream while spending some games on consequential branches that search found difficult. Section 4.2 explains how restart states are selected and how resignation is checked against games allowed to continue to their natural result.

3.4 Replay and materialization

After a game finishes, its searched positions become replay rows containing the position, legal actions, search policy, and outcome target. The workers publish complete trajectories; materializers convert them to a circular memory-mapped store. Policies stay sparse until a training batch is assembled. Admission checks prevent incomplete or rejected games from being counted as usable data.

The trainer sees a mix of broadly sampled positions and positions where search substantially changed the network’s move preference. Replay grows as more games arrive, while the amount of optimizer work is tied to newly admitted positions. Section 4.2 follows these distinct choices—where games begin, what becomes a row, and which rows recur in training—and gives the retained settings.

3.5 Training

Training minimizes errors in the search policy and the game outcome. Two auxiliary tasks use information from a completed game: predicting the next move’s searched policy and the remaining game length. They help train the shared net-

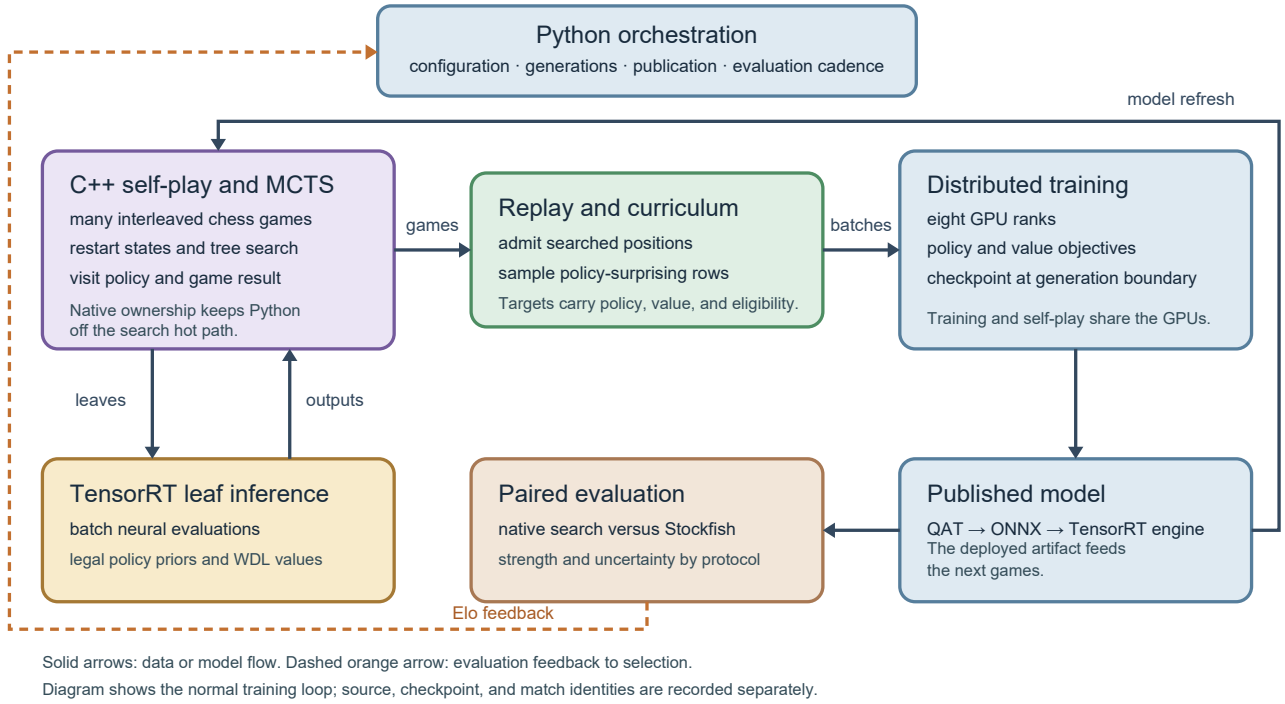


Figure 1: A published model guides batched native self-play; searched positions become replay targets for the trainer. Paired evaluation measures the published checkpoint and informs model-promotion decisions.

work but do not run during self-play. Section 4.2 explains when those future-dependent targets exist.

Eight persistent trainer processes, one per GPU, train in 500-step blocks with a global batch of 2,048 while some self-play workers continue producing games. A block followed by publication is a *generation*; its saved model is a *checkpoint*. Overlap matters: saving search work does not necessarily shorten a generation if training was already the limiting step. The adaptive-stopping test in Section 4.1 measures that distinction.

3.6 Progressive models and publication

Training starts with a small network: its speed produces more searched games while the learner has little use for extra capacity. A larger candidate trains on the same replay alongside the active model and takes over only after paired matches show it has caught up. Growing a trained model into a larger one was also tested, but is not the ordinary promotion path. Section 4.3 examines this choice; Chapter 7 specifies the plateau and match gates.

Publication saves a recoverable training checkpoint and a smaller inference artifact. The deployed path exports quantization-aware-trained weights to ONNX and refits a TensorRT engine for self-play and evaluation. The engine’s outputs are checked against the source model: a successful build alone does not prove that it plays the same moves. Chapter 6 explains the failure that prompted this check.

3.7 Evaluation

During training, fixed-position metrics and short paired-opening matches show whether learning is progressing. Larger matches at several search budgets test the final model; Table 1 in Chapter 2 reports their results. Each opening is played with both colors against a fixed Stockfish node limit.

Appendix B explains the rating calculation and gives supporting evaluation details.

The run preserves the configurations, engine and data identities, checkpoints, and evaluation records needed to interpret and reproduce those measurements. Appendix D lists the public and archived artifacts.

4 Research investigations

4.1 Spending search where it matters

More search makes an individual move stronger. With the same frozen network and opponent, raising the budget from 200 to 1,600 visits raised its match score from 0.318 to 0.748. Training poses a different question: could some moves use fewer visits so that self-play finishes more games without teaching the next model worse policies? Cheap moves might save computation, but the saved work matters only if it increases useful training data or shortens the learning cycle.

We tried fast/full searches, several ways to allocate or stop search according to position, and reuse through a graph or inference cache. The retained approach increases a fixed visit cap in stages and applies that cap to every recorded move. The experiments below distinguish four tests that can give different answers: immediate move strength, agreement with a deeper policy, strength of the next trained model, and throughput.

4.1.1 The fixed-budget baseline

The native engine performs policy-guided Monte Carlo tree search. The network supplies legal-move priors and a win/draw/loss value; PUCT selects leaves; neural evaluations are batched across active games; and values are backed up from the alternating player’s perspective. A fixed visit cap

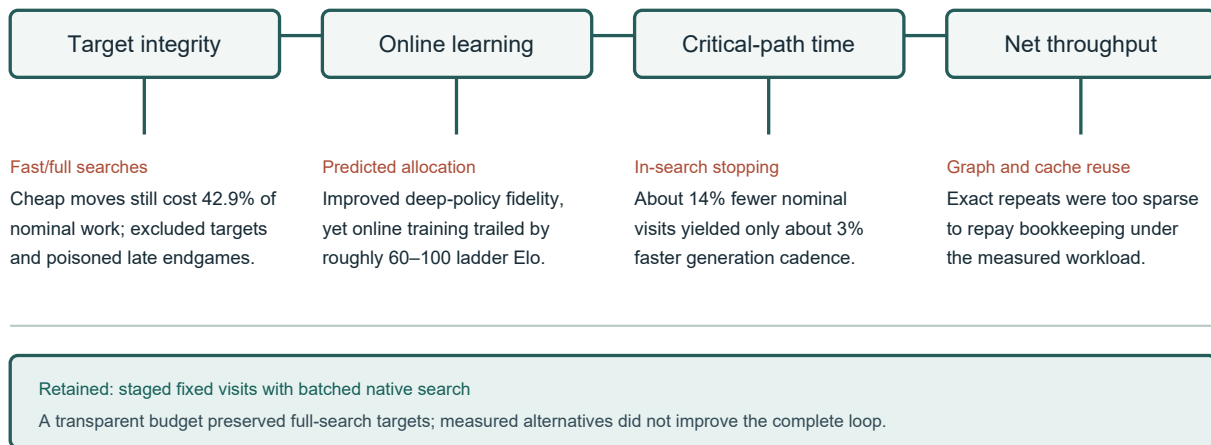


Figure 2: Fast/full search damaged target coverage and endgame semantics; predicted allocation improved target fidelity but lost online strength; in-search stopping saved mostly non-critical-path work; and exact graph or cache reuse failed to repay overhead in this workload.

gives each recorded move the same search budget within a training stage.

Against the same opponent, score rose from 0.318 at 200 visits to 0.537, 0.580, 0.662, and 0.748 at 400, 600, 1,000, and 1,600 visits. Across the better-resolved middle and deep part of the sweep, the fitted trend was about 81 Elo per doubling. Individual adjacent comparisons were noisy, so the overall trend is more credible than any single increment. Search was still changing its policy target too: at 600 visits, only 72.4% of positions selected the same best move as the network’s own 10,000-visit reference. These frozen-network measurements establish the value of search depth, not an optimal training schedule.

The cap increased in stages—300, 400, 500, 600, and finally 800 visits. These boundaries were not each optimized independently. A retained root may start a move with existing statistics, so its nominal visit cap can exceed the new work performed.

4.1.2 Why fast and full searches did not transfer

A scheme inspired by KataGo’s self-play methods [7] used cheap searches to advance most moves and full searches on a random minority, which alone became primary policy targets. The trade is attractive in 19×19 Go: long games make finished outcomes expensive, so cheap moves can supply more independent value targets. Chess games are shorter, their outcomes and positions are often clearer, and the value objective was already learning. The transferred recipe thus spent search on moves whose positions were discarded as primary training rows without showing that the extra completed games compensated for the lost policy-target density.

The cheap moves were not free. With a quarter of moves searched to 600 visits and the rest to 150, the cheap moves still consumed 42.9% of nominal search work. Nor were they isolated from training: they selected played moves, affected terminal outcomes and retained trees, could supervise a preceding row’s next-policy target, and influenced which positions entered the restart archive. “Not stored as a primary policy row” was never the same as “irrelevant.”

The mixed workload also interacted badly with batching.

Once the cheap searches finished, only the full-search minority remained. In a 512-game test this tail left about 128 active trees and filled only 86 of 320 available batch slots with one leaf per tree. Allowing four leaves per tree raised the average batch to about 268 and improved search throughput by roughly 20%, but it did so by making each search less serial. A policy intended to save compute thus created pressure to accept a search-quality tradeoff merely to keep the accelerator occupied.

Most importantly, a forced cheap-search tail made the endgame failure self-reinforcing. It removed searched endgame rows because cheap moves were not admitted as primary targets. A shallow estimate at the ply cap could then be propagated through many earlier rows of the same game. The original motive was reasonable—avoid filling replay with noisy positions from very long endings—but it also withheld the examples needed to learn conversion. One full search at the cut position replaced the heuristic bootstrap; later, removing the cheap tail restored searched endgame positions. Because multiple changes accompanied the observed recovery, the record does not isolate either repair as its sole cause (Chapter 6).

The retained design searches every recorded move to the scheduled cap. The decision follows from target coverage and throughput under this workload; a long-run matched-compute strength comparison was not preserved.

4.1.3 Three attempts to allocate search adaptively

The first attempt asked whether search could stop when the visit leader looked uncatchable. An audit of completed trees found apparent room: an optimistic reconstruction suggested that a rule requiring at least 75% of the cheap budget might remove 15.3% of cheap-search visits, or 6.38% of all nominal limits if full searches saved nothing. But the records contained final visit distributions, not the intermediate traces needed to identify when the leader first became safe. They also omitted enough state to reconstruct retained visits and forced-playout-pruned targets. The apparent saving was therefore an oracle-like opportunity estimate, not an observed stopping result.

A wider offline study found a deeper problem. The

marginal value of search was not monotone in visit concentration: very diffuse and already-decided positions gained little, while moderately concentrated but contested positions gained most. Simple concentration thresholds tended to stop in the very region where more search was useful. The rule-based approach was therefore declined before an online strength match.

The next allocator replaced the rule with a learned prediction, related in aim but not identical to dynamic simulation stopping [3]. An auxiliary head estimated, for several candidate budgets, the divergence between that budget’s policy and a deep-search policy. A calibrated corrector incorporated root observables, and a dual variable kept average spend near its target. Deep labels, replay persistence, model publication, native budget selection, safety gates, and telemetry were all implemented. Mechanically, the system worked. At approximately matched mean spend it captured about 23% of the available policy-divergence headroom; its mean target fidelity resembled roughly 1.18 times uniform search at 0.967 times the spend. It also behaved plausibly, assigning more work to contested positions.

Learning nevertheless deteriorated. Repeated online attempts trailed comparable non-adaptive training by roughly 60–100 ladder Elo. More than a third of positions received an average budget fraction near 0.36, and almost 9% received one eighth of baseline search. Those shallow policies were close to the network’s own prior yet were trained at full weight. The likely failure was therefore objective mismatch: closeness to a deep policy at the current position does not measure how much a target will improve the next network. That mediation remains a hypothesis, but the central result does not: the controller improved its stated proxy and still produced worse learning.

The final adaptive system moved the decision inside search, where a learned stopper could observe the evolving tree rather than predict difficulty in advance. Its decisive test started from the same checkpoint, optimizer, and rebuilt replay state for every arm. The most aggressive setting skipped about 14% of nominal search, and its internal credit-wait measurements changed in the expected direction. Yet generation cadence improved by only about 3%. Self-play overlapped the optimizer, so most of the removed search was slack rather than critical-path work. The paired strength differences, calculated as baseline minus stopper, were $+1.7 \pm 9.9$ Elo and -4.2 ± 10.1 Elo (standard errors) for the two settings: neither resolved a strength effect. At the observed learning rate, the cadence gain was worth only about one Elo over three hours, below the experiment’s resolution.

The three approaches failed for different reasons. The threshold rule lacked an identifiable safe signal; the predicted allocator improved policy fidelity but not learning; and the in-search stopper removed mostly off-critical-path work. In a non-overlapped or inference-bound learner, that last tradeoff may change.

4.1.4 Parallel leaves: buying latency with search quality

Batching across many independent games is the cleanest way to feed the accelerator, but the number of active roots eventually runs out. The engine can then keep several leaf traversals in flight from one root. Virtual reservations discourage those

traversals from selecting the same path, yet every selection is based on a tree that is missing the other in-flight results. Parallel search is therefore not serial MCTS executed faster. It exchanges fresher decisions for larger batches and lower latency.

The experiments exposed how easy it is to measure the wrong regime. An initial sweep suggested little cost per doubling, but its batch capacity was too small relative to the number of trees, so the configured per-root parallelism barely bound. A deliberately oversized-batch rerun activated the knob and estimated a steeper -6.4 ± 4.7 Elo per doubling. The interval still included a small effect, and the rerun was diagnostic rather than a production throughput configuration.

The tradeoff also depended on the total budget. At 1,000 searches, a terminal fixed-network sweep measured point losses of 19 Elo with four leaves and 45 Elo with sixteen leaves relative to serial search, while sixteen-way parallelism was substantially more damaging at 100 searches. The likely explanation is that a deep search will eventually visit more of the temporarily suboptimal leaves selected from stale state. However, only a few budget and parallelism combinations were measured, and the terminal strength curve itself uses different parallel counts at different depths. It is not a pure scaling curve from which a universal safe-parallelism law can be inferred.

The retained design uses bounded parallelism as a serving parameter rather than a free algorithmic speedup. The right setting depends on available independent roots, batch capacity, total search depth, latency requirements, CPU load, and memory. Older results from the fast/full batching tail do not directly prescribe the topology of the final all-full workload.

4.1.5 When a tree became a graph

Chess appears rich in transpositions: different move orders often reach the same board. The project tested whether Monte Carlo graph search, as in Czech, Korus, and Kersting [6], could share neural evaluations, descendants, and search statistics across those paths. Canonical nodes held shared state information while parent/action edges kept local PUCT statistics. The implementation handled backup corrections, cycles, rerooting, and pruning; an audit against the reference implementation corrected a missing first-link backup behavior before the final measurements.

The limiting fact was chess-state identity. Pieces and side to move are not enough: castling rights, en-passant state, the halfmove clock, and repetition-relevant history can change the legal result. Merging positions that differ on those fields would create an approximate algorithm with different game semantics. Under exact equality, most apparent board transpositions disappeared.

At ordinary budgets, verified links were absent or negligible. After the first-link correction, only 0.0249% and 0.1769% of neural evaluations were avoided at 1,000 and 10,000 searches, while the graph was 8.63% and 8.28% slower. At 30,000 and 60,000 searches, table hits rose to 2.37% and 3.46%, but avoided evaluations remained approximately 0.0001% and 0.0348%; throughput was still 7.06% and 5.76% lower. Structural counters confirmed that shared descendants and statistics were active. Exact reuse was orders of magnitude too sparse to repay the bookkeeping cost in this

implementation, so it was not taken to a final strength match.

4.1.6 Why inference caching found little to reuse

Inference caching asked a narrower question: if an identical encoded neural input reappears under the same model, can its policy and value outputs be reused without sharing search state? Two investigations answered complementary parts of it.

First, a real bounded, sharded cache was shared by the search threads within each self-play process. With eight processes per GPU, three search threads per process, and capacity for 1.5 million entries per process, its hit rate was 0.970%. Disabling the cache made game updates about 0.88% faster in the short stochastic comparison and reduced summed worker peak memory by about 7.12%. The implemented cache consumed memory without demonstrating a speedup.

A later audit measured the upper bound available to a wider cache before building one. An unbounded tracker shared across one search executor observed exact encoded inputs but deliberately evaluated every position. With diverse starts and production-style progression, repeats were 1.33% at 150 searches, about 4.24% at 800, and about 3.5% in the mixed workload; same-batch duplicates were essentially absent. Following retained trees for several moves raised the rate only to roughly 4%. The tracker itself cost about 3.65% throughput and grew without bound. A real cache would additionally pay for output storage, synchronization, eviction, and device transfers while retaining fewer entries.

The wider design was declined before implementation. Together with the implemented per-process cache, its opportunity audit showed little reuse available under this workload.

4.1.7 The retained search design

The selected system returned to fixed visits, but not to a bare or naive search. It uses PUCT, reduced-parent-value first-play urgency, root noise and temperature for self-play exploration, forced root playouts followed by pruning of visits that should not become policy supervision, batched native inference, a 0.99 per-ply value discount, and tree retention with 60% of visits carried across a played move. Every recorded move receives the current staged visit cap. The current settings are summarized in Chapter 7 and defined by the public configuration [10].

Search depth has direct strength and target-fidelity evidence, while parallel leaves have a measured quality/latency tradeoff. Root retention, forced playouts, noise, discount, and the PUCT constants complete the retained AlphaZero-style search recipe.

Fixed visits kept target provenance and search expenditure clear, while the tested alternatives failed to improve learning under this chess workload. Repetitive analysis or a non-overlapped learner could change the economics.

4.2 Getting more learning from each game

Once self-play finishes a game, the learner has to decide what it can learn from that game and when to see those positions again. It can start future games from an interesting branch, admit searched positions to replay, and revisit positions where search corrected the network’s move preference. These choices do different jobs: a restart creates new experience,

while replay sampling reuses experience already collected. A game stopped at the ply cap also needs a trustworthy value target because its actual outcome is unknown.

Figure 3 follows the route from a starting position to a training batch. We then examine how the system creates games, stores their searched positions, and decides which positions to revisit.

4.2.1 Starting games where information is likely

The start mixture tackles two coverage problems. Half of games begin after a uniformly selected zero to eight random legal plies. These prefixes diversify the opening cheaply; they are reconstructed in the recorded history but not searched or trained directly. Drawing zero plies also keeps some games at the ordinary initial position.

The other half begin from archived self-play states, falling back to random openings when a worker’s restart archive is empty. The archive favors positions with a meaningful unresolved alternative: they are not near the end of the source game or already decided, and search found a small set of plausible moves. A restarted game chooses an untried branch, reconstructs the prefix, then explores the alternative the source game did not play. Appendix D gives the eligibility thresholds.

The archive gives 30% probability to uniform selection. Otherwise it favors positions where search substantially corrected the network’s value estimate, while softening the priority so one extreme position cannot dominate. Age and capacity bounds remove old states, and exhausted positions leave the archive. Archives are local to workers.

Unlike a learned targeted-search controller [5], the archive uses observed search disagreement and untried branches. It creates new trajectories from those branches; the replay sampler below decides which existing positions recur during training.

4.2.2 What becomes a training position

Not every move in a recorded game has a search target. Random opening-prefix moves and the reconstructed prefix of a restart game were played or replayed to reach a starting position; they were not searched and create no replay row. A normal searched move does. A final search performed only to value a cut position can also become a row even though it selects no played action. Sparse policy storage retains at most 60 actions and records how much visit mass was discarded. These rules determine the data set before sampling or weighting can act on it.

4.2.3 Replay is both memory and clock

A replay window has to retain enough varied positions without teaching from very old play forever. A small FIFO forgets early policies quickly, but also forgets openings and endgames that have not recently appeared. A large window preserves breadth while keeping older targets alive. The system therefore begins with a small logical window and expands it as new games fill the store; merely reserving twenty million slots would not create twenty million distinct positions.

The physical memory map is preallocated once, while its logical capacity grows from 600,000 to 20 million rows. The

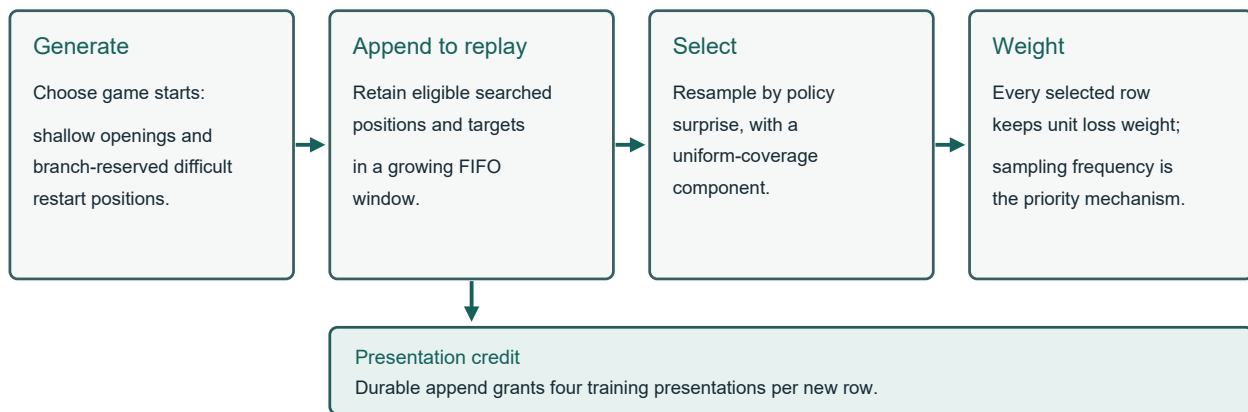


Figure 3: Starting positions determine which games are searched; admission determines which observations become replay rows; sampling determines which rows recur in training. Rows have unit loss weight, and newly admitted rows fund optimizer work only after durable replay append.

early window stays small when little data exists; later growth preserves more openings, endgames, and policy history. In an earlier campaign, playing strength continued to improve after fixed-dataset policy accuracy largely saturated, reinforcing the value of fresh and varied positions. Appendix D gives the full capacity schedule.

Replay reuse introduces a second tradeoff. The configured ratio is the number of optimizer presentations funded by each newly admitted row. In the retained setting, four presentations are credited per row; a 500-step quantum at a global batch of 2,048 therefore requires 256,000 newly appended rows. Credits are issued only after append and flush, and a persistent ledger prevents a restart from spending them twice.

Higher reuse funds more updates from each game; lower reuse gives each update fresher positions if the actors can supply them. Short controls at ratios four, 6.25, and eight showed similar strength despite different update rates. The selected ratio of four favors freshness. Because the comparison was short and other parts of the campaign also changed, it does not establish a separate final-strength gain. Appendix C gives the comparison’s scope.

4.2.4 Choosing which positions recur

Search is most informative when it substantially changes the network’s initial move preference. Within the live window, the sampler therefore favors rows with high *policy surprise*: divergence between search’s visit distribution and the network prior. Seventy percent of draws follow this signal, capped at 2.0 so extreme rows cannot dominate; the remaining 30% are uniform to preserve coverage. A row can recur across optimizer steps but is drawn only once within a global batch. This is related to prioritized replay [4], though it uses a different signal.

The optimizer learns from this prioritized distribution without inverse-probability correction. Policy surprise can also reflect search noise or target age, so the uniform component keeps broader coverage.

Loss weighting is separate. Each row has a positive sample weight which, after batch-mean normalization, multiplies the primary and eligible auxiliary losses. Rows in the se-

lected recipe use weight 1.0. Earlier replay could aggregate duplicate positions and encode multiplicity through weights; that established the mechanism, not a playing-strength gain from arbitrary weights. Thus the final system sees policy-surprising positions more *often*; it does not increase their per-draw loss weight. TD-error priority, recency weighting, and global deduplication remained proposals.

4.2.5 Ending games without corrupting their labels

Resignation can save a large amount of search, but a false resignation turns a drawable or winning position into an incorrect terminal label. A fixed value threshold was therefore replaced by continuing measurement. Twenty percent of games are designated at creation as continuation games and can never resign. They reveal the outcomes of hypothetical triggers for thresholds from -0.99 through -0.70. A trigger requires both the root value and the best visited child’s backed-up value to cross the threshold; evidence from capped games is excluded because their natural outcome remains unknown.

Over a rolling window, a candidate threshold needs at least 100 triggered continuation games and a one-sided 95% upper confidence bound on false non-loss no greater than 2.5%. The threshold may become more conservative immediately but relaxes by at most 0.01 per publication. An intentionally aggressive canary verified trigger journaling, persistence, and exclusion of a capped continuation. It tested the mechanism, not the safety of a production threshold. The retained system therefore calibrates and audits resignation continuously; neither zero error nor a separate Elo gain has been established.

A ply cap creates a harder target problem because there is no observed result at all. The project compared material heuristics, raw network value, values from adjacent searches, and a fresh search at the cut position. Playing games beyond the normal cap supplied later outcomes for evaluation. At the earlier measured checkpoint, the cut-position search achieved Brier score 0.444 and cross-entropy 0.756, compared with 0.491 and 0.851 for material divided by 39. At the later checkpoint the corresponding scores were 0.193 and 0.374 versus 0.388 and 0.707. Search-root sign accuracy exceeded 98% in both cohorts; calibration, not merely sign, distinguished

the targets. The retained worker therefore performs one full search at the actual cut position and uses its root value as the bootstrap. The root value becomes a soft win/draw/loss target, then materialization applies the configured per-ply blur. Appendix D gives the conversion.

The cut policy addresses the most damaging data failure found in the project. A cheap-search tail excluded late positions from primary replay while a shallow cutoff estimate supplied the value target for earlier rows in the same capped game. This plausibly reinforced weak conversion: fewer searched endgame examples, more capped games, and more weakly grounded targets. Replay eviction could remove the offending rows without undoing their effect on the network. Chapter 6 examines the measured incidence and repair. The general data rule is to consider cut-value provenance and policy-row eligibility together.

4.2.6 Knowing what each target means

An eventual game result is informative, but a position many moves before that result should not receive an equally sharp target. During materialization, the terminal WDL target is blurred toward uniform by 0.998 for each remaining game ply. During optimization, a scheduled coefficient eventually blends up to 10% of the stored search-root scalar into that discounted outcome. Search has its own 0.99 per-tree-ply discount, which changes backups, move selection, and the root value before replay is written. These mechanisms act at different points in the learning loop. The cut-position benchmark favors searched values over the tested material heuristic; it does not separately measure the ordinary 10% blend or either discount.

Future-dependent auxiliary targets require equally explicit eligibility. The retained next-policy target uses the sparse visit distribution from the following searched observation; it is unavailable if that observation does not exist. Remaining game length is known only when the game reaches a natural result or valid resignation, so every row from a cut game marks that target ineligible. The objective masks and renormalizes auxiliary losses over eligible rows rather than treating missing labels as zeros. Overfit tests showed that the multi-head targets were learnable, and replay audits checked censoring and offsets; their independent playing-strength contribution remains unmeasured.

Broader auxiliary bundles were set aside while diagnosing several simultaneous training problems, not because a controlled ablation found them harmful. Future-derived labels must come from complete trajectories and be masked whenever the future is unknown.

4.2.7 Fresh targets versus fresh positions

Reanalysis would spend search on fresher targets for existing positions instead of new states and outcomes. An earlier replay design implemented bounded synchronous re-search with source-bound target overrides, but no controlled study compared its learning return with fresh self-play. It disappeared with that replay schema. Reanalysis is therefore superseded infrastructure, not a negative efficacy result.

Model publication addresses freshness from the other side. Publishing every 100 optimizer steps imposed enough serialization, validation, and activation overhead to be rejected, but

no preserved matched strength curve gives that choice an effect size. The retained 500-step boundary trades target age against publication cost, and its wall-clock cadence depends on replay reuse.

4.2.8 Overlap without full asynchrony

Training proceeds in coordinated credit-funded quanta while half the actor processes continue self-play. A pause sweep found only a narrow, workload-dependent difference between balanced choices, so the retained fraction is an operational setting rather than a universal optimum. Publication, replay snapshots, and optimizer commits remain synchronized; fully asynchronous learning was proposed but not implemented. The throughput measurements and a corrected device-placement artifact are covered in Chapter 5.

4.2.9 Making the data trustworthy

Completed games enter a typed circular store through bounded, worker-owned materialization. Atomic writes, quarantine with a fatal rejection ceiling, immutable trainer snapshots, deterministic cross-rank sampling, and credit only after durable append keep targets and optimizer progress auditable under load. These are correctness and throughput properties, not direct chess-strength ablations; Chapter 5 covers the implementation.

4.2.10 What the retained curriculum establishes

The final data recipe combines staged replay growth, reuse four, policy-surprise sampling with a uniform component, unit row weights, shallow random openings, branch-reserved restart states, calibrated resignation, direct cut-position bootstraps, explicit target eligibility, coordinated publication, and partial actor/trainer overlap. Most of these choices coexist in one successful system and do not have one-variable online Elo estimates.

The cut-position comparison favors a searched value over the tested material heuristic, while the late-game poisoning incident shows what happens when target eligibility and value provenance are mishandled together. More presentations do not necessarily mean more information: reuse, freshness, diversity, and schedule pace move together. The resulting curriculum preserves a broad stream of searched positions and the meaning of each admitted target.

4.3 Choosing what the network predicts

A chess network must make accurate predictions quickly enough for search to finish millions of games. The move policy was the first design pressure: on a small model, a dense output layer can consume much of the parameter budget, yet a compact head still has to distinguish the legal moves of a position. We compared three ways of representing moves, then tested how the trunk, value prediction, model size, and quantization affected learning and inference speed.

4.3.1 Three policy representations

The same legal move can be assigned a score by a dense action list, by its move type on a spatial board, or by its origin and destination squares. All three families were implemented

against the chess move interface; the selected interface gathers 1,880 canonical actions. Figure 4 shows what each head asks the network to predict.

The **dense head** flattened a small spatial projection into 1,880 logits. Dense heads trained successful models, but on a small trunk their projection could dominate the parameter count. Variants changed projection width, spatial reduction, and final-map rank. A rank-96 variant reduced head size from roughly 484,000 to 207,000 parameters and was recorded as matching its immediate baseline; the result bundle is unavailable, so those variants cannot be ranked quantitatively here.

The **move-plane head** represented sliding directions, knight moves, and promotions as spatial move types. It could preserve board structure, but its experiments did not show an advantage over the dense control. In a short supervised comparison, the repaired plane head reached 2.1828 held-out policy cross-entropy after about 2,500 steps versus 2.0824 for the dense control, while its curve was still improving faster. An online plane-head trial was retired after slower learning and weaker play; its result artifact and exact representation have not been recovered. The plane approach is technically viable, but its relative playing strength remains unquantified.

The two implementations differed: one gathered 1,880 canonical logits from 76 planes (56 sliding directions, eight knight moves, and 12 promotions); the other exposed all 4,864 plane-square cells and normalized only legal moves. Native mapping checks covered 83,651 moves without a discrepancy. Neither the supervised comparison nor the surviving online record establishes the plane head’s eventual strength.

The retained **from-to head** preserves square structure without predicting a mostly empty plane tensor. It projects the 64 trunk squares into query and key vectors, scores all origin-destination pairs, and gathers the canonical actions through a fixed table. A separate projection adds queen, rook, and bishop offsets for promotions; en passant and the canonical castling encoding remain ordinary square pairs. On the controlled convolutional trunk, this head used 51,072 parameters rather than 483,680 for the dense alternative.

Holding that trunk fixed, the from-to head improved the held-out policy gap by 0.0298 nats (paired 95% interval 0.0285–0.0311). Spending the saved parameters on a wider trunk added only 0.0018 nats in the three measured cells; the missing fourth cell prevents a full factorial conclusion. The head cost approximately 1.9% of batch-512 forward throughput and 9% at batch 64. This short, single-seed teacher-data comparison establishes better policy fit; it did not measure a separate self-play Elo gain.

4.3.2 Board input and rule state

The selected model receives 52 side-to-move-canonical board planes: pieces, castling rights, en passant, checks, repetition, the eight most recent moves, material counts, and the fifty-move counter. Rule-sensitive planes keep positions with different legal or draw states distinguishable. File reflection is the only augmentation; it also mirrors action targets and exchanges kingside and queenside castling planes. Some earlier component comparisons used a 29-plane input, so their absolute scores are not input-matched to the final model.

4.3.3 Convolution, attention, and global context

The next question was whether attention could learn board-wide relationships better than a convolutional tower at the same practical serving cost. Both designs used a shared trunk for policy, value, and training-only heads. An earlier small model had spent a large fraction of its parameters on a dense policy head and a second policy-shaped auxiliary, making head choice especially important when comparing trunks.

The attention alternative treated the 64 squares as tokens, with learned row and column embeddings, pre-normalized self-attention, and GELU feed-forward blocks. No-bias, relative-offset, and input-dependent Smolgen-style attention biases were implemented; only no-bias and Smolgen have a preserved efficacy comparison. A CNN-attention hybrid was proposed but not built.

With bootstrap policy shape, head, runtime, and precision controlled, the convolutional trunk beat bare attention by 0.0060 nats on held-out teacher data. Smolgen gave the best attention cell, but changing the attention model’s dense head to from-to improved the held-out gap by a much larger 0.1573 nats. Attention also consumed more memory and served more slowly at the relevant batches. The tested convolutional design was the better choice for this workload; the result does not rule out attention in other chess systems.

Local convolution receives global context every second residual block: board-wide means and maxima from one quarter of the channels are projected back as biases on local features. Squeeze-excitation was another implemented context mechanism. Global pooling appeared to learn faster, with no clear final-strength difference, but the comparison artifact is unavailable. Its retention draws on that experience and the KataGo precedent, not an isolated chess Elo result. It also creates floating-point islands in the quantized graph: useful context and inference efficiency did not align perfectly.

4.3.4 Value and training-only heads

The outcome head predicts win, draw, and loss rather than a single scalar. Search converts this distribution to an expected value when necessary, while training and diagnostics retain the draw probability. A compact two-channel spatial reduction and 48-unit hidden layer was retained. A matched 32-channel probe added 97,020 parameters and reduced measured training throughput by 1.31%, while improving total loss by only 0.00309 in one short seed and slightly worsening WDL loss. This justified keeping the smaller head. The earlier scalar-to-WDL transition has no preserved isolated strength comparison.

Training also predicts the next move’s searched policy and normalized remaining game length, with loss weights 0.15 and 0.1. The next-policy target uses the following state’s own side-to-move action space, legality mask, and symmetry transformation. Both heads are removed from the serving artifact. Their target wiring, masking, gradients, and checkpoint behavior were tested, but neither objective has a long matched self-play ablation. Other auxiliaries were set aside while debugging potential interference, not because they were shown harmful.

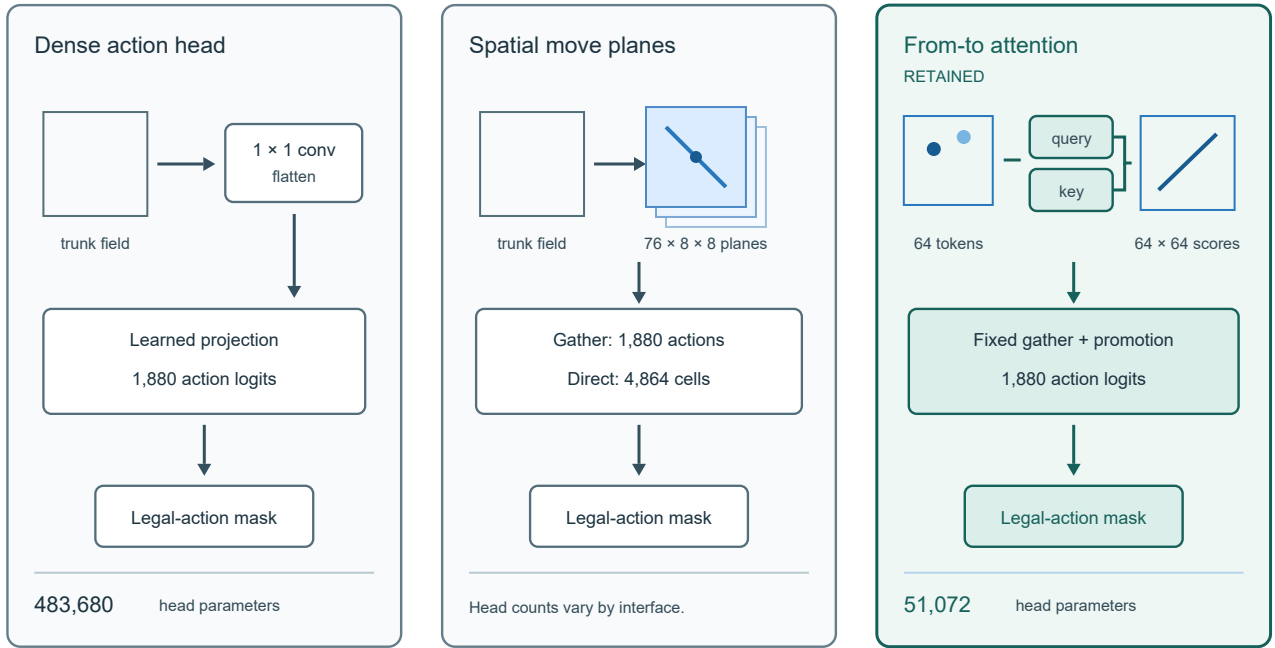


Figure 4: Dense heads project to a move list, move-plane heads predict spatial move types, and the retained from-to head scores square pairs before gathering legal actions.

4.3.5 Bootstrap and optimization controls

Initialization shapes the first search priors and, through them, the first self-play targets. In an initial architecture comparison, the attention policy put only about 0.11 probability mass on its top three moves while the convolutional control was effectively one-hot. Neither extreme represented chess knowledge, but each induced different data. The retained bootstrap uses architecture-appropriate initialization, deterministic construction, a small final policy projection, and calibration on 516 encoded positions toward a common policy concentration. Those positions set numerical scale without supplying supervised chess targets.

The retained quantization-aware optimizer is Nesterov SGD. Frozen-replay screens showed stable fitting and informed warm-up, learning-rate, and folding choices; they did not measure online Elo. The selected run ended at 408,500 optimizer steps, below its configured one-million-step folding trigger: the trainable model remained pre-fold, and folding was performed on a deployment copy. Delayed folding and higher post-fold rates helped short-horizon replay fitting in separate screens; they were not a phase of the reported training run. AdamW had trained successful earlier models and was superseded, not disproven. Frequent gradient clipping did not prevent learning in those screens, but its threshold was not shown optimal.

4.3.6 Quantization as an architectural constraint

Post-training quantization of the ordinary residual tower was fast and behaviorally unusable. Activation ranges grew from about 0.78 near the input to roughly 40–43 late in the network; depending on calibration, full-trunk INT8 preserved only 13.3–25.9% policy top-one agreement. Weight-only quantization was faithful but slower than TensorRT FP16. Quantization therefore became a network-design problem rather than

a final export switch.

A scaled pre-activation block bounded the learning graph, but its normalization, clipping, residual scaling, and requantization boundaries fragmented the compiled TensorRT graph. It expanded an 83-layer FP16 graph to 330 layers and an INT8 graph to 470 layers, with many reformats, yet still failed fidelity. The retained scaled post-activation block instead keeps the conventional sequence of convolution, normalization, capped activation, convolution, normalization, scaled residual addition, and capped activation. Activations are capped at six, and residual branches are scaled by the inverse square root of depth. The scale can be folded into the second convolution at export, which preserves more efficient compiler tactics.

This block learned normally under quantization-aware training. After continuation in the folded deployment topology, a production-sized smoke test reached roughly 135,000 INT8 positions per second, compared with 60,000 for TorchScript BF16 and 99,000 for TensorRT FP16. These are model-core rates, not end-to-end self-play rates. Folding only after training damaged agreement, and global context and the heads remain outside the INT8 trunk. The result is evidence that architecture and deployment had to be co-designed; it is not evidence that the scaled block plays better than an ordinary residual block in floating point. The fidelity and throughput measurements above distinguish the tested alternatives.

4.3.7 Progressive model sizing

KataGo provided the precedent [2]: start with a small, fast network, train the next size on the same data, and switch when it catches up. Early in self-play, extra model capacity may contribute less than the additional searched games a small network can produce. The measured small-model throughput supports that premise here, and the small-to-medium handoff worked repeatedly. There is no equal-cost fixed-size control,



The medium model is the reported checkpoint; the larger stage is a capacity hypothesis, not a demonstrated improvement.

Figure 5: The small model buys early self-play throughput, and a medium candidate trains on the same replay before paired-match promotion. The larger candidate may avoid catch-up with function-preserving growth, but the limited continuation did not demonstrate a strength gain; the reported checkpoint remains medium-sized.

so its exact Elo-per-currency contribution remains unknown.

Candidate start follows a stage-specific searched-Elo plateau; promotion instead requires two passing paired matches against the active model. The former loss-based gate promoted a candidate that was about 270 Elo weaker because extra catch-up updates made its training loss incomparable. The failure study in Chapter 6 explains that correction; Chapter 7 states the retained thresholds and promotion gate.

An independently initialized larger candidate needed substantial catch-up. Function-preserving growth avoided relearning the medium model’s function, but might also bias optimization toward its existing representation. The limited grown-model continuation reached parity without a clear strength gain. The reported model therefore remains medium-sized; choosing between catch-up and growth needs a matched-compute comparison.

4.3.8 Distillation and compact models

Two compression studies asked related but distinct questions. Teacher-output imitation trained compact students on a legal-masked policy and WDL distribution from policy-only teacher play; no search generated those labels. Increasing the dataset from one million to six million positions mattered more than a small capacity sweep. The strongest 1.33-million-parameter student trailed its teacher by 176.1 Elo at 25 searches each and by 38.4 Elo when it received the measured shallow equal-compute allowance of 58 searches. At 250 searches each the gap widened to 257.6 Elo. Deeper search amplified the better prior rather than washing out approximation error. Fixed-ply sampling gave the dataset only one side-to-move parity, and the teacher had a known conversion weakness, so these absolute gaps do not transfer to the final model.

Replay-target compression instead trained on sparse MCTS visits, outcomes, root values, and replay metadata from a frozen ten-million-row window. The selected 474,069-parameter student was 13.20 times smaller than its teacher. It trailed by 291.3 Elo at 64 searches each and by 166.2 Elo when its measured saturated serving advantage allowed 186 searches against 64. It reached statistical parity only at an equal-multiply-accumulate allowance of 850 searches, which ignored tree work, launches, and imperfect batching and was therefore not an equal-time result. The compact artifact was

useful, but it did not replace direct training. These were separate compression protocols, not direct comparisons with the final teacher.

A terminal compression check trained a 470,295-parameter student on a separate frozen 20-million-row replay snapshot. At 10,000 searches it reached 2,683 conditional benchmark Elo after roughly 7.5 epochs and 2,697 after roughly 23 epochs. The 14-Elo central difference lay well inside the match intervals, while held-out policy loss had become nearly flat. Tripling passes over this fixed buffer therefore produced no measurable playing gain. The longer student reached 2,873 conditional Elo at 100,000 searches, but the point is unbracketed and is not a calibrated deep-search headline.

These studies produced a practical compact artifact, but model size did not exchange mechanically for more search. Realized search multipliers were far below parameter or arithmetic ratios, and the teacher advantage often grew with search depth. Distillation was separate from the primary self-play training run.

4.3.9 Decision

The retained network combines the rule-complete 52-plane input, a shared convolutional trunk, periodic global context, the from-to policy head, a compact WDL head, and training-only next-policy and remaining-length objectives. Scaled post-activation blocks make the trunk compatible with quantization-aware deployment. Progressive sizing successfully exploited a small model before handing off to the medium model, while the value of the larger stage remains unresolved. Component tests support parts of this design, while final playing strength belongs to the assembled system.

5 From inference speed to learning speed

The model could learn only as fast as self-play supplied new games. Making one neural-network call faster helped, but a game still had to finish, its positions had to enter replay, and the trainer had to consume them. Figure 6 follows that path from local speed to useful training data.

5.1 Native ownership of the search loop

Each search leaf needs a board update, legal moves, encoding, and a neural evaluation. Dispatching that sequence through Python for every position made host work a bottleneck. The

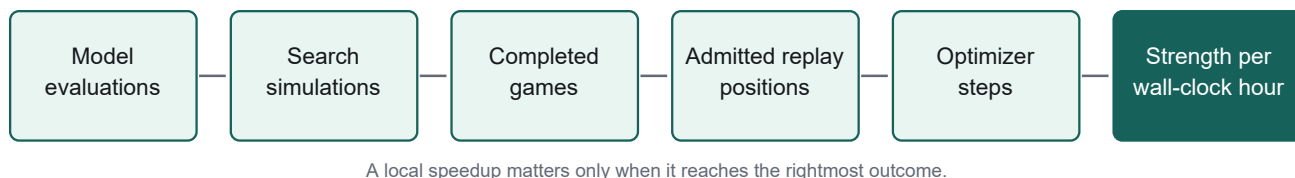


Figure 6: Search speed passes through game completion, replay admission, and optimizer work before it can affect playing strength. Each boundary has its own throughput measure.

production loop therefore keeps game rules, search trees, selection and backup, and asynchronous inference requests in C++. Python coordinates configuration, replay, training, publication, and evaluation outside that hot path.

That native loop can advance many games together: each process holds 512 roots whose leaf evaluations share one pre-allocated inference pipeline. Trees survive played moves, retaining useful search work. Four actor processes per GPU, one inference worker per process, batches of at most 320 positions, and two outstanding batches kept the GPUs busy in the retained topology.

5.2 Batching and host-side submission

Large neural batches help only while the host assembles, submits, and consumes them fast enough. Board encoding, persistent staging buffers, host-to-device copies, result processing, CUDA events and graph replay, process count, and inference-thread count were optimized as one submission path.

In a controlled 32-process self-play workload, the optimized path increased search throughput from 512,679 to 617,782 searches/s, a 20.5% gain, while reducing aggregate actor CPU consumption from 52.6 to 19.8 cores. Average inference batch size rose from 141 to 222 and the number of model calls fell by 24%. On the same node restricted to 24 CPU cores, throughput increased from 215,265 to 496,036 searches/s because reducing submission overhead allowed the previously starved GPUs to remain busy. The host-side gain was largest when CPU capacity was limited, showing that GPU search can be starved by CPU work.

Batch fill alone did not guarantee device saturation. With the TensorRT actor, reducing four processes per GPU to two lowered simulations/s by 39% even though both filled individual batches: concurrency overlapped CPU and GPU work around inference. Additional inference threads had the opposite effect, duplicating CUDA contexts and splitting requests into smaller batches. One inference thread per actor was the measured operating point.

Search parallelism is a separate batching lever. Multiple leaves from one root can fill a batch after other games finish, but virtual reservations make the search less serial and can reduce search quality. Chapter 4 therefore treats parallel search as an algorithmic quality-throughput trade, not a free systems optimization.

5.3 Inference runtimes and precision

Once the host could submit full batches, the inference engine became the next lever. A native TensorRT FP16 engine reached 75,889 positions/s against 40,716 for a TorchScript BF16 control in a matched benchmark, a 1.86x gain. Quantization-aware INT8 added a further 1.31x over Ten-

sorRT FP16 on the tested quantization-oriented network. Production-topology tests found INT8 gains of 14.4% for the smaller network and 39.1% for the medium network.

Speed was useful only if the deployed network still played the same chess. Post-training INT8 changed policy and value outputs too much, so the retained scaled post-activation blocks were trained with fake quantization. Backbone convolutions run in INT8; the start block, heads, and linear layers remain at higher precision. Publication recalibrates the checkpoint, exports an explicit Q/DQ ONNX graph, and refits a TensorRT template [8]. Policy and value outputs are then checked on encoded positions. Section 4.3 explains the block design; Chapter 6 shows why successful engine construction alone could not establish fidelity.

`torch.compile` was another attempted speedup. It accelerated eager batch-64 inference by roughly 27–33%, but fused TorchScript remained faster. In the tested eight-GPU training workload, compilation reduced throughput by about 18% relative to eager execution, while `bfloat16` autocast improved it by 9.2%. Compilation was not retained for production inference or training on this workload.

Model shape remained a systems variable even at similar parameter counts. Width and depth changed kernel efficiency, TensorRT tactics, and memory behavior discontinuously. Channels-last layout and cuDNN autotuning helped relevant CNN shapes, but no analytic parameter-count rule predicted the fastest network. Progressive model sizes were therefore benchmarked at their actual serving batch and precision rather than selected from FLOPs alone.

5.4 Replay materialization and training supply

Search work becomes useful training compute only after games finish and their observations pass replay admission. Completed games are atomically published, converted into typed columnar rows, and appended to a fixed-capacity memory-mapped replay. Materialization is partitioned before expensive conversion, bounds inbox scans and staging, quarantines malformed individual games, and fails loudly if systemic rejection prevents the learner from receiving data. This makes replay ingestion both a throughput stage and a scientific boundary.

The trainer reads vectorized batches from the memory-mapped store with pinned-memory prefetch and one persistent DDP rank per GPU. A loader benchmark on 2.5 million rows improved from 3,943 to 32,579 samples/s after compacting 5,000 small producer shards into 25 containers—8.26x faster and 44% above the measured trainer demand. In a later live interval, materialization appended 8,790 positions/s while accepted positions arrived at 1,365/s, leaving more than sixfold headroom. Materialization was not the bottleneck in

that workload.

A credit ledger links optimizer work to newly admitted positions. Each training quantum becomes eligible only after enough materialized data has arrived for the configured replay-reuse ratio. This prevents rejected games, ingestion stalls, or an accidental change in data reuse from appearing as normal optimizer progress. Persistent DDP ranks then avoid repeated startup and model construction costs. On the measured eight-GPU topology, a global batch of 2,048, bfloat16 autocast, and concurrent self-play produced 6,252 training samples/s; larger batches improved hardware throughput further but would also change update count and optimization semantics, so they were not adopted merely because the GPU benchmark was faster.

5.5 Overlapping self-play and training

Self-play, materialization, training, publication, and evaluation overlap without making the learner fully asynchronous. Optimizer work remains divided into explicit quanta, and actors switch models only at publication boundaries. A selected subset keeps producing games during training while the rest pause. This uses idle capacity without losing the identity of the model and data behind each quantum.

The overlap fraction is a measured compromise. With no actors running, the trainer processed 25,275 samples/s. Keeping 8, 16, or all 32 actors active reduced trainer throughput to 21,492, 17,139, and 9,210 samples/s, while simultaneously producing approximately 506,000, 606,000, and 742,000 searches/s. When both training time and the remaining self-play wait were included, the estimated complete-quantum times were 117.1, 112.9, and 111.2 seconds. The broad result was that moderate overlap improved total cadence, while running every actor nearly doubled the trainer quantum for only a small additional end-to-end gain. The retained half-active policy sits near that measured tradeoff rather than maximizing either trainer or actor throughput alone.

The useful overlap fraction depends on search cost. As visit budgets and model size rise, self-play becomes more expensive relative to training; actor settings measured at the beginning cannot simply be extrapolated to later stages.

5.6 Where throughput becomes useful

Faster simulations did not translate mechanically into training throughput. A production-shaped TensorRT INT8 benchmark ran search 1.86 times faster than its floating TorchScript control. In a separate live stage, admitted replay positions arrived 23.3% faster at 400 visits than at 600. The two comparisons changed different things, so their speedups cannot be multiplied. Game length, publication pauses, and complete-game replay admission all stand between simulation rate and optimizer cadence. In the live stage, admitted-position rate predicted that cadence.

The relevant systems measure is therefore how quickly the loop produces completed games, admitted replay, and optimizer progress—not simulation rate alone. Appendix C records the local comparison boundaries.

6 Three failures that changed the method

Three plausible shortcuts failed at different boundaries: replay admission, inference-artifact fidelity, and model promotion. In each case, a convenient success signal measured something other than the behavior the system needed.

6.1 Late-game target poisoning

The self-play design once reduced search near the end of long games. The intent was to keep noisy positions from exceptionally long endgames from dominating replay. If those positions would not become training targets, spending full-search compute on them seemed wasteful, so the tail used cheap searches instead. Only full-search positions were admitted as primary training rows, however, so the cheap-search tail selectively removed the conversion phase from the training set. The resulting policy was weakest exactly where it needed to turn an advantage into a terminal result. Games wandered to the ply cap more often, and the cutoff then assigned one terminal value to every earlier training row in the trajectory.

This created a closed feedback loop. Missing searched endgames produced weak late-game play; weak play produced drawn-out, unconverted games; the cutoff supplied an unreliable target; and that target reinforced the same policy and value errors. During the most affected early interval, roughly one ply in seven was structurally ineligible for training, 27–32% of games reached the cap, and about 36–38% of admitted rows inherited the value assigned to a cut game. The corruption was therefore not confined to the last position: the final target propagated back through the entire recorded trajectory. By the time the active replay buffer looked healthy, the damaging rows had already been evicted, while their effect on the weights and subsequent self-play distribution could remain.

The repair happened in two stages. First, the cutoff stopped relying on the material heuristic and obtained a value from one full search at the final cut position. A controlled continuation study supported that choice: on 2,282 early cut positions, the searched root value reduced Brier error from 0.491 to 0.444 and cross-entropy from 0.851 to 0.756 relative to the material target. The same ordering held on a later set of 1,144 positions. Later, the forced cheap-search tail was removed altogether, returning properly searched endgame positions to replay. The second change closed the data hole that a better terminal value alone could not repair. Because other late-game changes were bundled around the same period, neither step receives an isolated Elo credit. The evidence establishes the failure mechanism and target-quality improvement, not a one-variable strength estimate.

Search eligibility, replay admission, and terminal-value provenance must be designed together. Weakening or discarding one part of a trajectory can make the model least competent there; subsequent self-play then reinforces the same blind spot.

6.2 A mechanically valid but semantically invalid TensorRT refit

The deployment pipeline builds a structural TensorRT template and refits it with each new quantized checkpoint. One template was built from a checkpoint whose clipped activation quantizers had many numerically equal scales. At high optimization levels, TensorRT optimized the engine around those equalities. Training later separated the scales, but the refit API still accepted every replacement weight, reported no missing weights, and returned success. The resulting engine was structurally valid and semantically wrong.

The defect was visible only when the published engine was treated as a chess model rather than as a successfully serial-

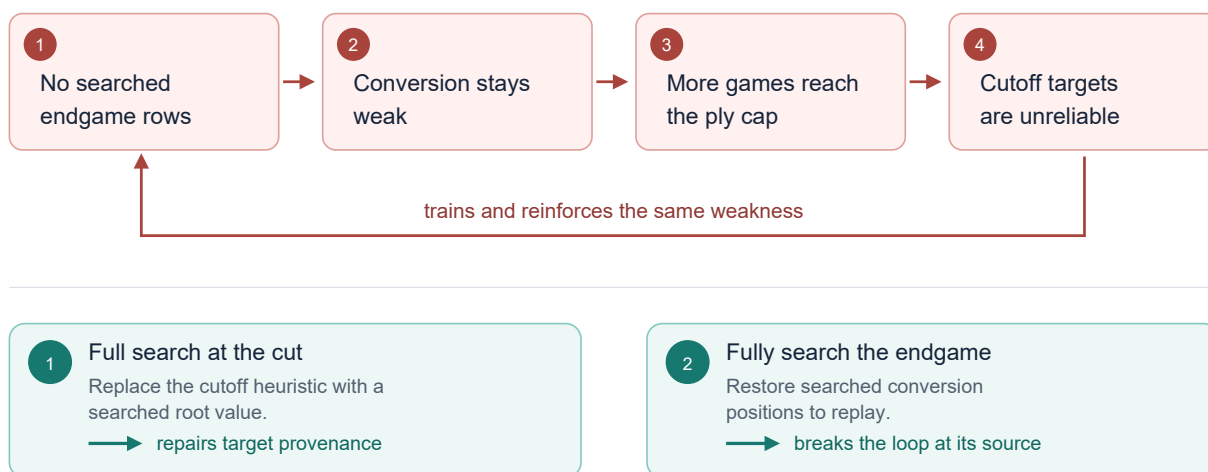


Figure 7: The cutoff repair fixes target provenance; restoring fully searched endgames breaks the feedback loop at its source.

ized object. On 516 real positions, the faulty template produced legal-policy KL near 1.05 and repeated refits of the same inputs changed individual logits by 10–15. Building at a lower optimization level, or first making the template’s scale constants distinct, reduced legal-policy KL to about 0.0012 and made repeated refits deterministic. The failure required three conditions: equal scales in the build source, optimization level four or higher, and a later refit that made those scales unequal. Tests with identity batch normalization and zero biases rejected constant folding as the cause. These probes isolated the refit-template defect.

Template construction now separates equal quantization scales before optimization and uses a safer default optimization level. More importantly, deployment publication no longer treats deserialization, complete weight accounting, or refit success as evidence of model equivalence. It evaluates the engine on real encoded positions and records legal-move top-one agreement, legal-policy KL, and WDL error against the source artifact. Those checks cover the semantics that search actually consumes: which legal move the policy prefers, how its probability mass changes, and whether the value distribution remains faithful.

A compiler or refitter can meet its mechanical contract while violating the model’s behavioral contract. Published artifacts therefore require representative behavioral probes alongside their graph, template, runtime, and refit manifest.

6.3 Promotion from incomparable training losses

The progressive controller originally promoted a larger candidate when its smoothed training loss caught up with the active model. Loss looked like a cheap proxy for readiness: it was already measured during training, whereas a full-search candidate match consumed additional evaluation compute and time. That comparison ceased to be meaningful once the candidate received extra optimizer quanta over the same replay distribution. More presentations gave it a systematic advantage on replay loss without proving equal playing strength. The loss gate promoted a candidate whose deployed artifact passed inference-fidelity checks but was still about 270 Elo weaker in play.

Promotion now measures the property that publication re-

quires. The candidate plays paired head-to-head matches against the active deployment artifact and must score at least 0.48 in two consecutive completed evaluations. A failing score resets the sequence; a failed or cancelled match contributes no evidence. Candidate-start timing, extra catch-up training, and promotion are separate controls. Function-preserving growth is likewise a separate attempt to remove the larger model’s initial relearning deficit, not a substitute for the playing-strength gate. Chapter 7 states the retained candidate-start and promotion procedure.

Training loss remains valuable for optimization diagnostics. It is not a promotion criterion when the compared models have seen different numbers of presentations or when the artifact intended for deployment can be tested directly.

7 Integrated chess recipe

The final chess recipe balances three needs. Smaller networks make early searched self-play affordable; the learner later moves to a stronger network when progress slows. Games continue to produce searched positions through the endgame, while restart states and replay sampling return informative positions to training. Quantization-aware training lets the model that learns also serve the INT8 search system. These choices ran together on one node with eight NVIDIA GeForce RTX 4070 SUPER GPUs and 80 logical CPUs. The fully expanded `chess-final-config.yaml` [10] supports new runs; Appendix D records the secondary settings and distinguishes this living recipe from the frozen result.

7.1 Network and learner

The configured network ladder contains 12×128, 14×160, and 19×176 residual CNNs. Each has capped scaled post-activation branches and global-pooling context in every second block. A from-to policy head predicts chess moves, while a two-channel WDL head predicts outcomes. Training-only heads also predict the next searched policy and remaining game length; deployment removes them.

Policy and value are the primary training targets. Terminal outcomes are discounted per ply, with a small search-root-value blend introduced over training. If a game is cut short, the searched root value substitutes for an unknown terminal

result, and its unknown remaining-game-length label is censored. Appendix D gives the weights and discount factors.

Training runs in 500-step quanta on eight ranks with a global batch of 2,048, using Nesterov SGD and bfloat16. The trainable network uses quantization-aware training (QAT) from the beginning. Its parameters remain pre-fold: folding, recalibration, ONNX export, and TensorRT refitting happen on a deployment copy. This protects the optimizer’s model while producing the INT8 artifact used for searched play. Appendix D gives optimizer, calibration, and learning-rate settings.

7.2 Game supply and replay

Thirty-two actors run self-play, four per GPU, with 512 games interleaved in each process. Half the actors on each GPU pause during a training quantum; the rest continue searching. This overlap keeps new games arriving without surrendering the whole node to either training or self-play. Chapter 5 measures the throughput tradeoff.

The fixed search budget rises from 300 to 800 visits per move. Fixed visits remained in the recipe after adaptive allocation and learned stopping failed to improve wall-clock learning. Appendix D gives the search constants and native inference batch settings.

Games begin approximately equally often from up to eight random legal opening plies and from recent restart states. Restart candidates are screened for value, remaining length, age, and branchable visit mass; an unavailable restart falls back to a random opening. Game caps grow from 150 to 250 plies, and greedy move selection begins later as training matures. Resignation is calibrated against no-resignation games so that the upper bound on a false nonloss stays below 2.5%; 20% of games are designated as such safety continuations at creation.

Replay capacity grows through ten stages from 600,000 to 20 million live rows. Each admitted position funds four training presentations. Sampling keeps 30% uniform probability while otherwise prioritizing bounded policy surprise, so difficult positions are revisited without excluding the wider game distribution. Completed games become durable rows before training credit is committed.

7.3 Sizing and promotion

The smoothed 64-search ladder signals when an active model reaches a stage-specific plateau. Its successor then trains on a replay snapshot with a separate catch-up learning-rate clock. Candidate-versus-active games, rather than training loss, decide promotion: extra candidate presentations had made earlier loss comparisons misleading and had admitted a much weaker model. Appendix D gives the catch-up ratio and match gate.

A separate capacity study grew the trained 14×160 network into 19×176 while preserving its function, then recovered deployment fidelity through QAT. The larger continuation reached parity without establishing a stronger plateau, so the selected checkpoint remains the 14×160 model. That continuation was too short to determine why further growth did not help.

7.4 Measuring progress during training

Every 20 minutes, 50 paired openings supported a 64-search ladder against a three-rung adaptive bracket of fixed-node Stockfish 13 opponents. A policy-only ladder used the same openings and opponent framework. These measurements guided progress and candidate timing; the terminal matches reported in Chapters 2 and 8 are a separate evaluation.

8 Final training and evaluation results

The selected 14×160 chess model has 6.32 million parameters and reaches **3,251 benchmark Elo at 100,000 searches per move** in the paired Stockfish 13 evaluation introduced in Chapter 2. This chapter follows its 2.5-day training trajectory, compares it with the previous chess recipe, and shows what additional search and a much smaller distilled model achieve.

8.1 Training progress

The 64-search training ladder rose from 798 to 2,372.2 over 2.5 effective days, with a peak of 2,407.6 in that window. Figure 8 places this trajectory beside four earlier chess campaigns. The plot ends with the selected training run at 2.5 days; Appendix B explains the comparison windows.

The plotted curves make the progression visible, but the previous baseline and final run originally used different ladder estimators. Recomputing their plateaus on the same three-rung estimator gives 2,283.9 and 2,358.0 benchmark Elo: **about +74 Elo** for the final recipe. This compares complete training recipes, not an isolated contribution from any one change. Appendix B gives the transfer calculation and its uncertainty.

Checkpoint 1026 came from the strongest region of the 64-search ladder. A later function-preserving 19×176 continuation recovered its parent’s strength but did not establish a higher plateau. Appendix A describes the selected training trajectory and checkpoint identity.

The selected checkpoint followed **408,500 optimizer steps** in 817 training quanta. At a global batch of 2,048, that is **836,608,000 training presentations**. Self-play completed **3,249,647 games**, materializing approximately **209.15 million net positions** over the same lineage; replay held **16 million live rows** at selection. Thus the learner saw about four training presentations per admitted position. Appendix A shows the trajectories and counting boundaries.

At a conservative 100 searched plies and roughly 600 simulations per game move, 3.25 million completed games correspond to approximately 195 billion search simulations. Most require a neural-network evaluation.

Across 480 small-model quanta, median trainer throughput was 16,977 samples/s; across 337 medium-model quanta it was 11,194. The 2.5-day final training cost **\$43.20** in node rental at \$0.72/h. Appendix A shows the throughput traces; the cost excludes the separate experiments and evaluations.

8.2 Playing strength across search budgets

Search transforms the same trained network from **1,658 benchmark Elo without search** to **3,251 at 100,000 searches per move**, a difference of 1,593 points on this benchmark. Table 1 in Chapter 2 gives all ten paired-match rows; Figure 9 shows the selected rating at each budget, with the alternate tested opponent rung visible beside it. Gains con-

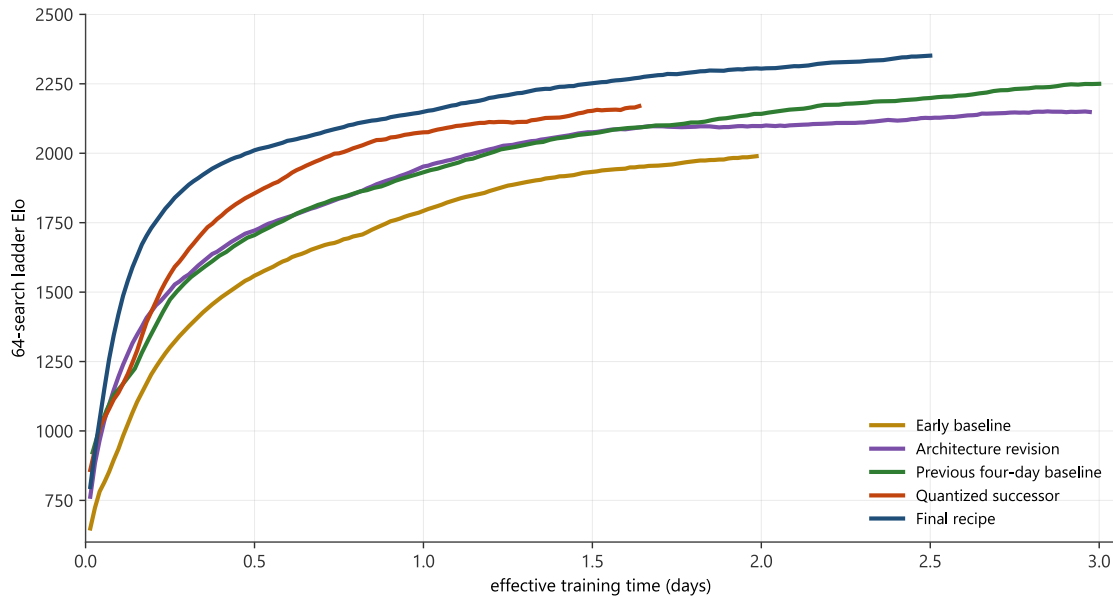


Figure 8: Smoothed 64-search training ladders across five chess campaigns. The final curve ends at 2.5 days and the preceding baseline at 3.0 days; the matched comparison below uses a common rating calculation.

tinue through the deepest measured point, though each later increase in search buys a smaller increment of Elo.

At the deepest budget, the 100,000-node and 200,000-node Stockfish opponents imply 3,247 and 3,251 benchmark Elo, respectively. Their four-point agreement supports the headline result. A 100,000-search move takes roughly five seconds of thinking time on an RTX 4070 SUPER with the current setup.

Both students trained on the same frozen 20-million-row replay snapshot. Appendix B gives the match counts and evaluation details.

Parallel search trades time for strength The measured strength curve uses one parallel search at 100 and 1,000 searches per move, four at 10,000, and sixteen at 100,000. A controlled 1,000-search sweep against the same 20,000-node opponent shows why: one parallel search measured 2,823 Elo, four measured 2,804, and sixteen measured 2,778. Their central differences of 19 and 45 Elo are smaller than the overlapping match intervals, while match time fell from about **18.3 minutes** to **3.6** and **1.6 minutes**, respectively. Four-way and sixteen-way parallelism made those matches roughly five and eleven times faster.

At only 100 searches, sixteen-way parallelism reduced the central strength estimate by 235 Elo. The available points suggest that a fixed parallel count becomes less costly as the total budget grows, but they are too sparse to specify a safe budget-by-parallelism frontier. Appendix C gives the controlled operating points.

8.3 Distilling a smaller player

A 470,295-parameter student—13.4 times smaller than the teacher—reached **2,697 [2,640, 2,753] benchmark Elo** at 10,000 searches after 110,000 training steps. The shorter 36,621-step student reached **2,683 [2,637, 2,731]** under the same opponent and search setting. The 14-Elo central difference lies within match uncertainty, and held-out policy loss had nearly flattened. At 100,000 searches, the longer student reached **2,873 [2,819, 2,935]**, although only one opponent rung was run there.

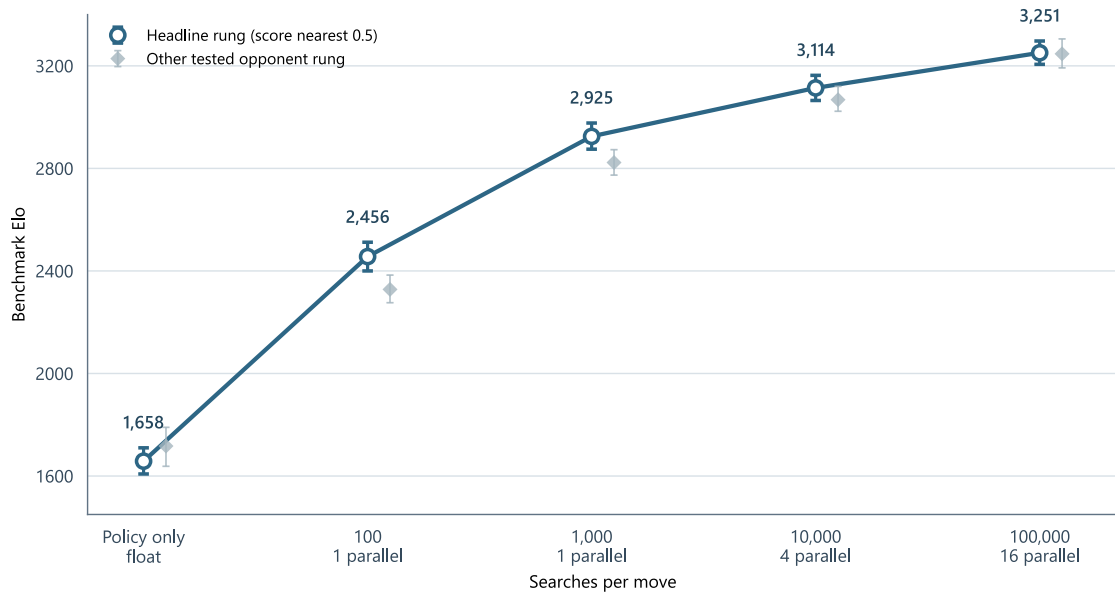


Figure 9: Connected points use the opponent rung with score nearest 50%; pale diamonds mark the other tested rung, and bars show 95% match-bootstrap intervals. The categorical horizontal axis separates policy-only play from 100, 1,000, 10,000, and 100,000 searches per move; its spacing does not represent compute.

9 What the evidence does not establish

The complete run measures the assembled system, but it cannot assign an Elo gain to each ingredient. Several component studies used one seed, a short horizon, frozen replay, or a local proxy. They help explain mechanisms and screen choices, but do not establish long-run online gains. Historical policy-head and global-context comparisons without complete artifacts remain qualitative. The larger network reached parity in a limited continuation; more training would be needed to separate optimization difficulty from limits in capacity, replay, or target quality.

The final ratings come from 100-game paired matches against fixed-node Stockfish 13 and a historical node-to-Elo calibration. The bootstrap intervals cover match sampling, not uncertainty in that calibration. Opponent rungs disagree at shallow budgets; policy-only and searched play use different inference artifacts; and deep-search parallelism varies with budget. Figure 9 therefore shows attainable operating points under the stated protocol, not an isolated search-scaling law or a rating against unrestricted engines.

The rejected methods' economics also depend on workload. Exact graph reuse and inference-cache hits were too rare in diverse chess self-play; adaptive stopping saved simulations but barely shortened training under actor/trainer overlap. A repetitive analysis service or non-overlapped learner might behave differently. The student comparison uses a different deployment backend from the teacher. The \$43.20 rental figure covers only the selected training path, not total project spending. Finally, a reproducible run should preserve its recipe, artifacts, match protocol, and statistical results; proprietary drivers and TensorRT versions preclude a promise of bitwise-identical weights.

10 Conclusion

In 2.5 days on one eight-GPU node, an AlphaZero-style chess system trained from random initialization produced a 6.3-million-parameter model measuring **3,251 benchmark Elo at 100,000 searches per move** against a fixed-node Stockfish 13 ladder. Without search, the same model measured **1,658 benchmark Elo**.

The result belongs to the assembled learning loop: compact structured policy prediction, a shared convolutional network, progressive small-to-medium sizing, searched self-play with restart-state and replay selection, and an INT8-capable architecture. Native tree ownership, batched inference, TensorRT, and distributed training made the necessary volume of searched data feasible. The component studies explain the design choices without claiming an independent Elo gain for each one.

The negative results sharpen that conclusion. A learned allocator produced better deep-policy fidelity but worse on-line learning. A stopper saved simulations but barely reduced wall-clock cycle time. Exact graph and neural-cache reuse were too sparse for their overhead in this chess workload. More parallel leaves sped service while spending some playing strength, especially at shallow budgets. Three failures exposed the coupling that those local metrics miss: omitting searched endgames poisoned value targets, a mechanically successful TensorRT refit changed model behavior, and incomparable training losses promoted a much weaker candidate.

The strongest general lesson is to measure the whole loop. For every proposed saving, ask how many additional reliable positions it creates, how quickly the learner absorbs them, and whether the deployed artifact plays better. The larger model recovered its parent's strength without surpassing it in the available continuation, while the compact student preserved substantial strength without matching the teacher. Longer train-

ing, different targets, and more capacity remain open paths rather than demonstrated improvements.

References

- [1] Silver, D. et al. (2017). *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. [arXiv:1712.01815](#).
- [2] Wu, D. J. (2020). *Accelerating Self-Play Learning in Go*. [arXiv:1902.10565](#).
- [3] Lan, L.-C. et al. (2021). *Learning to Stop: Dynamic Simulation Monte-Carlo Tree Search*. [arXiv:2012.07910](#).
- [4] Schaul, T. et al. (2015). *Prioritized Experience Replay*. [arXiv:1511.05952](#).
- [5] Trudeau, A. and Bowling, M. (2023). *Targeted Search Control in AlphaZero for Effective Policy Improvement*. [arXiv:2302.12359](#).
- [6] Czech, J., Korus, P., and Kersting, K. (2020). *Monte-Carlo Graph Search for AlphaZero*. [arXiv:2012.11045](#).
- [7] KataGo project (release 1.17.1). [Self-play training methods](#).
- [8] NVIDIA. [TensorRT engine refitting](#).
- [9] Meloni, M. (2021). *Stockfish and Lc0, test at different number of nodes*. [Calibration benchmark](#).
- [10] Braun, B. [Advanced Techniques in Chess Engines: source code and release documentation](#).
- [11] Braun, B. [AlphaZero chess model artifacts and model card](#).
- [12] Braun, B. [Live chess demonstration](#).

A Training diagnostics

The figures in this appendix describe the accepted training lineage through the selected checkpoint. They do not include the reverted INT8 conversion, the invalid capacity-promotion branch, or the later limited continuation of the larger model. A quantum comprises 500 optimizer steps at a configured global batch of 2,048. The selected checkpoint follows 817 contiguous quanta and records 408,500 optimizer steps and 836,608,000 presentations. The published checkpoint number, 1026, is an artifact identifier, not the number of quanta included in this selected-lineage plot. It was selected from checkpoints near the strongest 64-search ladder region and was the last there with complete model, optimizer, QAT, ONNX, and TensorRT artifacts retained. The plotted lineage excludes reverted INT8 and capacity-promotion branches.

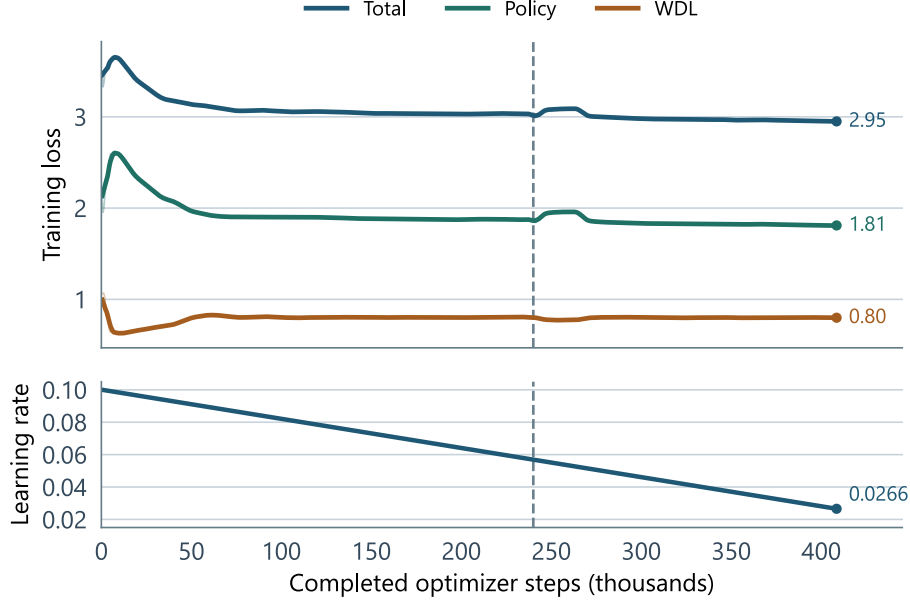


Figure A.1: Total, policy, and WDL training losses are shown as raw traces and 11-quantum moving averages, with final values labeled directly. The dashed line marks the small-to-medium active-model transition at 240,000 optimizer steps. Loss levels across model stages are optimization diagnostics, not paired playing-strength measurements. From 300,000 steps to selection, total loss falls only from about 2.98 to 2.95, while WDL loss stays near 0.80. The active learning rate fell from about 0.10 to 0.0266 by selection.

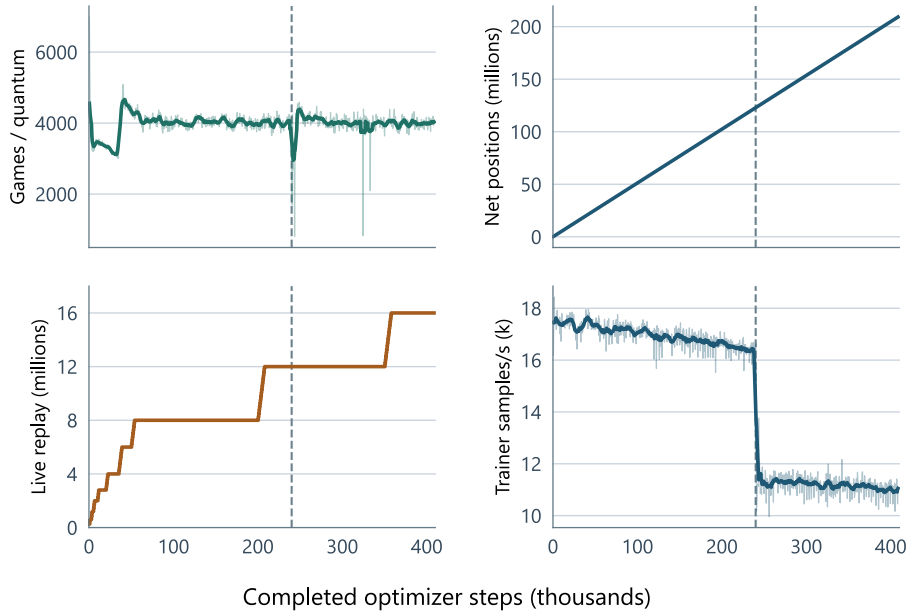


Figure A.2: Four panels separate per-quantum completed games, cumulative net materialized positions, live replay occupancy, and training samples per second on the same optimizer-step axis. The separate replay scale exposes its capacity steps: 209.15 million net positions were materialized, while 16 million rows were live at selection. The small model's median measured trainer throughput was 16,977 samples/s across 480 quanta; the medium model's was 11,194 across 337. Model shape, schedule, and concurrent workload differ, so the gap is not an isolated model-size effect. All panels describe the selected lineage, whose coordinator counters sum to 3,249,647 completed games.

B Evaluation detail

B.1 Fixed-node match protocol

The ten teacher matches are shown in Table 1. Every row used 100 games from 50 paired openings, once from each colour. Stockfish 13 used one thread and 1,024 MiB hash. Its fixed node limits were assigned the historical benchmark-Elo anchors of 1,700 at 1,000 nodes, 1,890 at 2,000, 2,220 at 5,000, 2,470 at 10,000, 2,700 at 20,000, 2,960 at 50,000, 3,100 at 100,000, and 3,230 at 200,000 [9]. The displayed 95% intervals bootstrap match outcomes while holding these graph-read anchors fixed; they exclude calibration uncertainty. The headline for each candidate budget uses the tested opponent whose observed score is closest to 0.5. Searched games used the INT8 TensorRT artifact; policy-only games used the corresponding float TorchScript export.

For score s against an opponent anchor R , benchmark Elo is $R + 400 \log_{10}(s / (1 - s))$. Each 95% interval resamples the 50 colour-swapped opening pairs 10,000 times, then transforms the 2.5th and 97.5th percentiles of the resulting match scores with the anchor held fixed.

The two deepest-search anchors agree within four Elo. At 100, 1,000, 10,000, and 100,000 searches, the harder opponent rung reads 128, 102, 46, and 4 Elo higher than the easier rung. Ideal transitive Elo would give the same estimate from both. Draw behavior, anchor calibration error, matchup effects, and sampling noise are possible contributors, but the project did not isolate the cause. Choosing the score closest to 0.5 limits extrapolation. The ladder is an attainable operating curve, not a constant-parallelism search ablation.

B.2 Matched-estimator comparison

The cross-campaign headline does not subtract the displayed 64-search curve endpoints. A retrospective plateau comparison on the same three-rung estimator placed the previous baseline at 2,283.9 and the final recipe at 2,358.0 benchmark Elo, a difference of +74.1. Restricting both plateaus to the clean report-figure windows (3.0 and 2.5 days) gave +75.8. The earlier baseline’s original series used a single-rung fit. A +2.7 Elo estimator transfer, summarized from 35 paired observations in the operator recap, brings it onto the three-rung scale. The underlying paired-observation table is not preserved with the report; approximately ± 15 Elo is a sensitivity allowance for this transfer, not a bootstrap interval. This limitation does not apply to the terminal match intervals above, which come from their game records.

For orientation only, the preceding four-day baseline is plotted through 3.0 days, retaining 143 observations and ending at 2,265.4 Elo before its noisy terminal interval; subtracting this from the final endpoint gives 106.8 Elo. The final endpoint also sits 348.2 Elo above the early baseline’s terminal observation. Neither visual subtraction measures cross-campaign strength because the original estimators differ. The smoothed report plot uses bias-corrected 0.95 exponential moving averages. Its final curve retains 180 observations through 2.5 days; the preceding baseline retains 143 through 3.0 days. Later points are excluded because they belong to subsequent experiments or an unreliable terminal interval.

B.3 Distilled student controls

Both students had 470,295 parameters and trained on the same frozen 20-million-row replay snapshot without QAT, separate from the teacher checkpoint’s 16-million-row live window. They were evaluated with TorchScript and four parallel searches, whereas the searched teacher used INT8 TensorRT. The 36,621-step student scored 31/33/36 against 20,000 Stockfish nodes at 10,000 searches: 2,683 [2,637, 2,731] benchmark Elo. The 110,000-step student scored 35/29/36 under the same protocol: 2,697 [2,640, 2,753]. At 100,000 searches, the longer student scored 59/28/13 against 20,000 nodes for 2,873 [2,819, 2,935]. That deep point is unbracketed because the planned 50,000-node match was skipped; it is not an architecture-only comparison with the INT8 teacher.

C Supporting search and systems comparisons

These controls make the denominators behind the local speedups explicit. They describe measured operating points, not additive contributions to the final model’s Elo.

C.1 Actor overlap and training supply

On the eight-GPU production-shaped workload, keeping more actors alive during a training quantum traded trainer throughput for ongoing search. The complete-quantum estimate includes the training time and remaining self-play wait; it is not the trainer-only duration.

Table C.1: Actor overlap and training supply

Active actors	Trainer samples/s	Concurrent searches/s	Complete quantum (s)
8	21,492	506,000	117.1
16	17,139	606,000	112.9
32	9,210	742,000	111.2

With no actors active, trainer throughput was 25,275 samples/s, but no new self-play games arrived during training. Half-active operation was retained near the measured tradeoff. The optimal fraction can change with visit budget, model size, and

hardware contention.

C.2 Replay-reuse controls

Short controls at reuse ratios four, 6.25, and eight remained matched at their shared evaluation boundaries despite different update rates. They lasted about 90 minutes, and their full raw curves are no longer preserved. The completed campaign also changed replay capacity, optimizer, objective weighting, and inference. The selected ratio of four is therefore a freshness choice, not an isolated final-strength estimate. Configured presentation credit need not equal the observed number of presentations per distinct replay position. Because evaluation, publication, visit schedules, and replay growth advance at quantum boundaries, changing reuse also changes their wall-clock pace.

C.3 Parallel-search operating point

At 1,000 searches per move against the same 20,000-node Stockfish opponent, parallel leaves substantially shortened the recorded match wall time while reducing measured playing strength. The two timing sources give ranges, not statistical intervals.

Table C.2: Parallel-search strength and wall time

Parallel leaves	Benchmark Elo	Match wall time (min)	Elo relative to serial
1	2,823	18.1–18.5	0
4	2,804	3.4–3.8	−19
16	2,778	1.3–1.8	−45

The relative penalty was much larger at only 100 searches: sixteen parallel leaves cost about 235 Elo. These few budget-by-parallelism points do not define a universal safe concurrency curve. The terminal strength curve varies parallelism with budget and must be read as an operating curve.

C.4 Negative search and reuse controls

The learned pre-search allocator captured about 23% of available deep-policy-divergence headroom at nearly matched spend but trailed non-adaptive online training by roughly 60–100 ladder Elo. A later in-search stopper skipped about 14% of nominal simulations, yet improved generation cadence by only about 3% under actor/trainer overlap. Its paired strength differences, calculated as baseline minus stopper ($+1.7 \pm 9.9$ and -4.2 ± 10.1 Elo, standard errors), did not resolve an effect.

Exact graph search avoided only 0.0249% and 0.1769% of neural evaluations at 1,000 and 10,000 searches while being 8.63% and 8.28% slower. A bounded inference cache shared inside each self-play process had a 0.970% hit rate; disabling it made game updates about 0.88% faster and lowered peak worker memory about 7.12%. A wider unbounded repeat tracker observed at most about 4% reuse in the tested production-like workloads and itself cost throughput. These are workload-specific negative findings, not general limits on graph search or caching.

C.5 Throughput comparison boundaries

The 1.86x TensorRT INT8 versus TorchScript search comparison used a production-shaped 400-visit actor workload, but different checkpoint weights; it is a backend-and-checkpoint result, not an isolated precision effect. The 23.3% replay-admission gain compared separate live 400- and 600-visit stages that also differed in checkpoint and actor scheduling. Absolute systems rates vary with CPU quota, GPU power, PCIe and NUMA layout, runtime versions, model shape, batching, and concurrent training.

D Reproducibility and release boundary

D.1 Two reproducibility targets

Reproducing the current system and reproducing the reported result require different starting points:

1. **Recipe reproduction:** use the current fully expanded `chess-final-config.yaml` as the supported entry point.
2. **Result reproduction:** use the frozen source revision, resolved config hash, manifest, checkpoints, engines, datasets, and archive recorded for the final result.

The recipe may evolve; the reported result remains fixed.

D.2 Expanded chess recipe settings

The architectural settings summarized in Chapter 7 include a key-size-128 chess from-to policy head and a two-channel WDL head with a 48-unit hidden layer. The training-only next-searched-policy and remaining-game-length heads have loss weights 0.15 and 0.1. Primary policy and value losses each have weight 1.0. Terminal outcome targets are discounted by 0.998 per ply; the search-root-value blend rises from zero to 0.1 over its configured schedule, while search backup uses a separate 0.99 per-ply discount. At a capped game’s final position, a searched scalar value v is converted to a soft WDL target with $r = 1 - |v|$: win, draw, and loss receive $\max(v, 0) + r/3$, $r/3$, and $\max(-v, 0) + r/3$, respectively.

Each 500-step training quantum uses eight ranks processing 256 positions each, for a global batch of 2,048. Nesterov SGD

uses momentum 0.9, weight decay 0.0001, and gradient clipping at norm 1.0. The learning rate warms from zero to 0.1 over the first 1,000 optimizer steps and then follows the configured linear schedule to 0.01. Training uses bfloat16 and persistent trainer processes; `torch.compile` is disabled. QAT calibration uses 516 real evaluation positions and is refreshed at every publication boundary.

The 32 self-play actors run four per GPU, with 512 interleaved games per actor. Native inference uses batches of 320 with two outstanding batches per worker. Search uses exploration constant 1.5, reduced-parent FPU with reduction 0.2, forced playout coefficient 1.5, and Dirichlet epsilon 0.25 with alpha 0.3. Restart-state selection retains a 30% uniform component. Eight materializers convert completed games into the fixed-layout memory-mapped replay store; training credit is committed only against durable admitted rows. Restart positions require at least 15 plies remaining in the source game, absolute root value at most 0.8, and two or three leading actions covering 85% of visit mass. The played branch is marked used, and reservations keep workers from claiming the same alternative concurrently. Replay’s logical capacity grows through 0.6, 1.2, 2.0, 2.8, 4, 6, 8, 12, 16, and 20 million rows. Restart selection is 30% uniform and otherwise weighted by the square root of value correction, defined as half the absolute difference between searched root value and raw network value. Age and capacity bounds remove old states, and the archive is local to each worker.

The successor network trains on a captured replay snapshot for an average of 1.5 optimizer quanta per active-model quantum, with its own catch-up learning-rate clock. The configured match gate requires the successor to score at least 0.48 in two consecutive paired evaluations. The fully expanded config [10] specifies the stage-specific ladder-plateau thresholds and window.

D.3 Result identity and provenance

The frozen local result record identifies these layers of provenance:

- Git revision and clean/dirty state, resolved YAML and SHA-256, and dependency-lock hash;
- operating image, software and driver versions, GPU, CPU, RAM, disk, and engine versions and archive hashes;
- evaluation dataset and opening-suite identities, hashes, raw matches, aggregates, commands, and interval method;
- run manifest, approval, coordinator logs, TensorBoard events, and resource telemetry;
- replay schema, capacity, occupancy, and reconciled volume counters;
- checkpoint and inference hashes, ONNX and TensorRT provenance, calibration positions, and fidelity reports;
- one digest covering the fetched archive or a checksummed artifact manifest.

The published INT8 ONNX has SHA-256 `d634abacae3c874eac6ded89f6af861eb81b509da638b5ad710587b1a08be658` at the immutable model-repository revision `dc8fcccb67ab5ec9e36267a165a9700b7dbf55f` [11]. The source release’s evidence index [10] records additional artifact hashes. The frozen training-source revision, resolved configuration, large run archives, and some exact evaluation inputs remain in the local result archive. The public recipe and model support inspection and a new run, but are not yet a self-contained package for exact-match reproduction of the reported experiment.

D.4 Reproducing the software

Local setup and validation begin in the public source-code release [10]. Production nodes are provisioned by `deployment/setup_remote.sh`, which installs locked dependencies, builds the Release extension, installs pinned evaluation engines, and runs engine smokes. Run lifecycle operations go through `deployment/run_control.sh`.

Original project code and documentation, including this report, are available under the MIT License in the source release [10]. The published final model artifacts carry the same license in the model repository [11]. External dependencies, reference sources, and third-party data retain their own terms.

D.5 Reproducing evaluation

Use the preserved checkpoint and inference artifact rather than re-exporting it with a newer toolchain. Reuse the same paired opening suite, colors, opponent binary, node limit, threads, hash, candidate search budget, parallelism, batching, and adjudication rules. Report every game and recompute aggregates independently.

For latency, separate:

- isolated model-forward throughput;
- saturated many-position search throughput;
- single-game interactive latency.

Only compare like with like. Appendix B states the terminal opponent, openings, game count, and inference artifact.

D.6 Reproducing plots and tables

The plots derive from archived JSON, CSV, TensorBoard, or manifest data. Figure inputs and extraction methods are retained beside the final evidence archive. Derived tables preserve the raw columns needed to recompute totals, rates, Elo transformations, and uncertainty intervals; live-dashboard values are not treated as publication data.